ARI 410/510 Project Phase 2a:
Task Competition Launch (with project groups)

University of Michigan-Flint

Fall 2026

(See course Canvas page for due date)

1 Project Phase Overview

In this part of the project, you will work together with your group to analyze the annotations that your classmates completed for your data, compute agreement, determine ground truth labels, and build baseline models. You will then launch your task as a competition with a leaderboard so that other teams can submit models to it in Phase 2b. The main deliverable will be a presentation (on the assignment due date) of your task and competition page. Setting up the competition page will likely be the most time consuming step, so make sure to budget time for that.

Right now: confirm your annotations are completed. if you have not received all of your assigned annotations, please contact the course instructor (email or Discord) as soon as possible. If you do not contact the instructor, it will be assumed that you now have all of the annotations that you need to proceed with this phase of the project.

2 Steps to Follow

1. Compile Annotations from Classmates

Gather the annotations provided by your classmates for your dataset. This may involve compiling data from various formats (e.g., spreadsheets or JSON) into a structured table where each row corresponds to a data point and each column represents an annotator's label for that data point.

2. Compute Agreement Between Annotators

Once you have gathered the annotations, calculate the agreement between the annotators. Agreement metrics help assess the consistency of the annotations. Some commonly used metrics include:

- **Cohen's Kappa**: For pairwise agreement between two annotators.
- **Fleiss' Kappa**: For agreement among multiple annotators.
- **Krippendorff's Alpha**: Useful when there are missing annotations.
- **Percentage Agreement**: A simpler measure showing how often annotators agree. Only use this in combination with at least one of the chance-corrected metrics.

These metrics will help you determine if the annotations are reliable or if there are significant disagreements among annotators. Read a bit more about each metric to see which one is most suitable for your dataset and compute the metric. You may use preexisting libraries to do this for you.

3. Determine Ground Truth Labels

Next, decide on the ground truth labels for your dataset. These will be the final labels used for training and evaluating your models. Some possible methods for determining ground truth include:

- **Majority Voting**: The most common label is taken as the ground truth. This requires having more than 2 annotations per data point. If there are only 2 annotators and you need a third vote, you could label some of the data points yourself to provide the tiebreaker.
- **Weighted Voting**: Assign weights to annotators based on experience or reliability.
- **Adjudication**: Manually review data points where annotators disagree and discuss between the team members until a resolution is reached on the correct label.

Be prepared to share how you computed the ground truth labels and explain why you chose that method. Keep a list of which data points the annotators disagreed on, this may be useful later.

4. Analyze Data

With the ground truth labels in hand, you should analyze the distribution of labels in the dataset. Create a visualization to show the amount of data with each label. Be prepare to report on: whether the classes are balanced or imbalanced.

5. Set Up a Competition Page for Your Task

Create a competition page to benchmark models for your task. This platform will allow you and others to submit models and compare their performance on a leaderboard. We recommend a Kaggle Community Competition (choose "Machine Learning Competition"), which is free. Kaggle's setup guide walks through every step. Codabench is also a great platform if you prefer it, but setup generally takes more work and has a steeper learning curve. Steps include:

- **Visibility**: Set your competition to **Private**, so it is only accessible through its invitation link.
- **Deadline**: Launch your competition by the due date of this phase and set its end date to the end of Phase 2b. That is when the private leaderboard is revealed. Kaggle usually shows times in UTC, so double-check that the end time you set is the one you intended.
- **Task Definition**: Describe your task and provide a clear description of the dataset.
- **Split data**: Choose a training, validation and test split for your dataset. Tip: make sure there are enough examples of each of your classes in the test set so you can evaluate models' performance for each class (e.g., do not put all of the "cat" images in the training set and leave none for the test set — you may choose to use stratified sampling to accomplish this).
- **Prepare the files**: Kaggle expects four CSV files, each with an ID column first:
  - `train.csv`: inputs and labels (put your validation data here too, or in a separate file)
  - `test.csv`: test inputs *only*, no labels
  - `solution.csv`: the test labels, plus a column named `Usage` that marks each row `Public` or `Private`
  - `sample_submission.csv`: an example of exactly what participants should upload (e.g., ID column plus a prediction column)
- **Keep test labels hidden**: `solution.csv` is uploaded only as the competition's answer key and is never shown to participants. Never post your test labels anywhere else, including GitHub, Google Drive, or Discord. If they leak, the leaderboard is meaningless because someone could train their model directly on the test set.

- **Public vs. private leaderboard**: Rows marked `Public` are scored on the leaderboard everyone sees during the competition (so they can see how well they are doing). Rows marked `Private` are scored on a hidden leaderboard that becomes the final ranking at the deadline . You might choose to, e.g., split your test set roughly in half between public and private examples. If your test set is small, you may not split exactly in half, but think about how many items each set will get.
- **Evaluation Metric**: Specify which metrics (e.g., accuracy, F1-score) will be used to evaluate submissions, and explain why that metric fits your task (hint: look back at Step 4).
- **Baseline Model**: Submit your baselines from Step 6 and mark them as "Benchmark" submissions so they appear on the leaderboard as a reference point.
- **Rules**: Participants have to accept your rules before they can download the data, so write a short rules page. At minimum: the data is for this course only, and predictions must come from a model. Labeling the test set by hand is not allowed. You may set other rules about the types of models participants should use or not use, whether they can pretrain their models on other data beforehand, etc.
- **Data License**: State who may use the data and for what (e.g., "for use in ARI 410/510 only"), and follow the terms of wherever the data came from.

Before launching, test your competition: submit your own `solution.csv` as a prediction file and make sure it gets a perfect score, then submit `sample_submission.csv` to make sure it gets scored. Then save the invitation link from Kaggle's launch checklist.

6. Create Baselines

Develop baseline models to compare against more advanced models. This sets a few references for your classmates to compare against. This will include:

- **Dummy Baselines**: These are simple models that provide a baseline level of performance. There are a few options, and you should choose whichever of these two gives the best performance:
  - **Random Baseline**: Randomly assigns labels to the data.
  - **Majority Baseline**: Always predicts the most common label in the dataset.
- **Trained Model**: Train at least one simple model (e.g., Logistic Regression, Random Forest, Feedforward Neural Network) on your dataset.
- **LLM Baseline**: In your proposal, you argued that your task is not trivial for a frontier LLM. Now test how true this turned out to be. Give a frontier LLM (e.g., ChatGPT, Claude, or Gemini) at least 50 test items zero-shot (no labeled examples in the prompt) and compute its score with your metric. You do not need to pass your entire test set through if that becomes too time consuming or expensive, but do at least 50 data points. You do not need to pay for this. The simplest way is the free chat version: paste your instructions and about 10 numbered items per message, ask for one label per item, and start a new chat for each batch. If you would rather script it, the Gemini API has a free tier (get a key from Google AI Studio). Report which model you used, the exact prompt, and how you sent the items. This one does not need to go on the leaderboard unless you ran all test set items through.

Remember to include the dummy and trained baselines with your initial task launch on the leaderboard.

7. Present Your Task in Class

You will present your task and findings to the class on the assignment due date. Presentations must be 5 minutes or fewer and should cover:

- **Dataset Overview**: Remind us about the dataset and task, what kind of data you collected, and why it is interesting or important. Show an example of a data point.

- **Annotation Process**: Describe your annotation setup, how you computed agreement, and determined the ground truth.
- **Data Analysis**: Show visualizations of label distribution.
- **Competition Setup**: Briefly introduce your competition page and explain how it works, including the submission format.
- **Baseline and Model Performance**: Present results from the dummy baseline, the trained model, and the LLM baseline.
- **Conclusion**: Make a brief pitch for why others in the class should make a submission to *your* task. You may even offer some kind of prize(s) but are not required to.

Make sure your presentation is clear, concise, and well-organized, with visuals that help communicate your results.

3 Deliverables

Make a post on Discord before the start of class in #project-competitions) containing:

1. Your slides or a link to your slides
2. The link (or invite link) to your competition page

And be prepared to make your presentation during class on the due date.

4 Working Together: If Your Team Is Having Trouble

Use this section if any member of your team feels that your team hasn't been adequately following its contract in the phases so far, or during this phase.

First step: talk to your teammates

Raise the problem with your teammates directly, using the communication channels and response times in your team contract. If your contract doesn't set a response time, 3 business days is a fair maximum to wait for a reply. Everyone is busy, but that's long enough.

Be specific about what was agreed and what hasn't happened. Keep a record of your messages and your own work (commits, files, annotation counts). Before the instructor gets involved in any team issue, they will ask to see that you did this first.

Option A: your teammates are willing to work it out

If everyone is willing to try, keep working together. You don't need to submit anything. Agree among yourselves on who owns what for this phase and exactly when everything will be completed.

If commitments are missed *again*:

1. The team sends the teammate a written notice (template below), copying me. It names the missed commitments and the deliverables and dates expected.
2. The team may ask to split. The removed student(s) then hosts their own variant track, under the same rules as Option B.

Option B: talking didn't work

If your teammates won't engage, or you can't agree on how to work together, you may split. Send me a split request as soon as you know this needs to happen. A split made now carries through Phases 2b and 3. If the split is proposed within 1 week of the current phase's deadline, by default it will not take effect until the start of the next phase. This is to ensure nobody is unexpectedly left on their own with only a few days to complete the phase. However, if all team members agree to split immediately, that request will be honored.

Each part of the team proposes a variant of your task that uses the same annotations. A variant has to change the prediction problem, not just its name. Examples:

- Binary vs. fine-grained labels
- A subset of the categories
- A different unit of prediction (span vs. whole document)
- Predicting where annotators disagree instead of the label they did agree on

The member or members who did the work on the original task keep it. The rest of the team proposes the variant. If it isn't clear who did the work, the instructor will decide based on the evidence available.

With instructor approval, the variants run as separate tracks of one shared task (e.g., Task 3A and Task 3B), the way many real shared tasks do, on the same timeline as everyone else. Everyone's annotations stay in the shared dataset. Other students compete on one track of their choice. After the split, each part of the team completes the remaining phases on its own.

Notice template (suggestion)

```
To: [teammate]   CC: [instructor]
Re: [Team name] contract commitments

Under our team contract, you agreed to [commitment(s)]. As of [date],
the following haven't been completed: [specific items, with dates].
We contacted you about this on [dates/channel] [and haven't received
a response].

For this phase, you're responsible for:
- [Deliverable], due [checkpoint date]
- [Deliverable], due [date]

If these aren't delivered by [final deadline], we'll ask to restructure
the team, and you would complete the remaining phases individually.
We'd prefer to keep working together. Please reply by [date] to confirm.

[Team members' names]
```

5 Other questions?

Training my model is taking too long! Your trained baseline does not need to be a strong model; it is a reference point for everyone else. Develop on a small subset first. For text or images, pretrained embeddings with logistic regression on top (like Lab 3) is a perfectly good baseline and runs fine on the free tier of Google Colab.

Do I have to pay for an LLM for the LLM baseline? No. See Step 6: the free chat versions are enough for 50 items. One caution: free versions may use what you paste to train their models. Check your data's terms of use before pasting it, and if your data includes anything about real people that they wouldn't want shared, talk to me first.

Do other students need an account to submit? Yes, participants need a free Kaggle account. Kaggle does not show hosts participants' email addresses, so when you join another team's competition, use a team name that includes your project team's name/number. Otherwise nobody can tell who submitted what.

Something on Kaggle isn't working. Who do I ask? Kaggle does not give hands-on support for community competitions. Check the setup guide first, then ask on Discord. Another team has probably already run into the same problem.

I have a different question! Please feel free to reach out via email or Discord at any time. I will populate this section with commonly asked questions for future semesters.